

Học Sâu (Deep Learning)

Chương 3: Giới thiệu TensorFlow, PyTorch, JAX và Keras

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Lịch sử phát triển các Framework Học sâu
2. Phân tích Sức mạnh của từng Framework
3. Mối quan hệ giữa Keras và các Backend
4. Bài toán cốt lõi: Bộ Phân loại Tuyến tính
5. Ứng dụng nâng cao (Tổng quan kiến trúc)
6. Tổng kết

Kỷ nguyên khởi nguyên (Trước 2009)

- Các nhà nghiên cứu phải tự viết tay logic tính toán đạo hàm (thường bằng C++).
- GPU programming gần như bất khả thi do chưa có kiến trúc hỗ trợ tốt.
- Năm 2006, NVIDIA phát hành CUDA, nhưng chủ yếu phục vụ mô phỏng vật lý chứ chưa phải Machine Learning.

Sự ra đời của các Framework đầu tiên (2009 - 2015)

- **Theano (2009):** Framework đầu tiên hỗ trợ tự động tính đạo hàm (autograd) và tính toán GPU. Là "tổ tiên" của mọi framework hiện đại.
- Cuộc thi ImageNet 2012 làm bùng nổ sự quan tâm toàn cầu tới Deep Learning, thúc đẩy các framework như Torch 7 (Lua) và Caffe (C++).
- **Keras (Đầu 2015):** Ra mắt với tư cách là thư viện Deep Learning cấp cao, dễ sử dụng, được xây dựng trên nền Theano.

Kỷ nguyên hiện đại (Cuối 2015 - Nay)

- **TensorFlow (Cuối 2015):** Do Google phát triển, hỗ trợ huấn luyện quy mô lớn (distributed computation).
- **PyTorch (Cuối 2016):** Do Meta (Facebook) phát triển để đối trọng, nổi bật với tính linh hoạt (Dynamic Computation Graph).
- **JAX (2018):** Một hệ thống gọn nhẹ từ Google kết hợp Autograd và trình biên dịch XLA, tối ưu vô cùng mạnh mẽ cho GPU/TPU.

Ba tính năng cốt lõi của một Framework

Để huấn luyện Deep Learning hiện đại, mọi framework (TF, PyTorch, JAX) đều phải giải quyết tốt 3 bài toán:

- ➊ Khả năng tự động tính đạo hàm cho các hàm phức tạp (Auto-differentiation).
- ➋ Khả năng tính toán số học tensor cực nhanh trên GPU/TPU thay vì CPU.
- ➌ Khả năng phân tán tính toán trên nhiều thiết bị (Multi-GPU/TPU) hoặc nhiều máy tính.

Tương lai của hệ sinh thái Frameworks

- Ngôn ngữ lập trình: Python thống trị hoàn toàn.
- Chúng ta đang sống trong thế giới Đa-Framework (Multi-framework): TF, PyTorch, JAX đều có chỗ đứng riêng.
- **Giá trị của Keras 3.0:** Giúp nhà phát triển chỉ cần viết mã một lần (Keras API), có thể linh hoạt chuyển đổi backend chạy trên bất kỳ framework nào (TF, Torch, JAX).

- Là một hệ sinh thái mã nguồn mở (open-source) đồ sộ của Google.
- **Hệ sinh thái Enterprise (Doanh nghiệp):** Gồm TFX (quản lý workflow), TF Serving (triển khai API), TF Lite (di động), TensorFlow.js (trình duyệt).
- Phù hợp cho việc đưa sản phẩm AI ra môi trường thực tế (Production-grade).

Tensors và Variables trong TensorFlow

Tensors là hằng số (không thể gán lại giá trị).

```
1 import tensorflow as tf
2 # Khởi tạo hằng số
3 x = tf.ones(shape=(2, 1))
4 y = tf.zeros(shape=(2, 1))
5 z = tf.constant([1, 2, 3], dtype="float32")
6
7 # Khởi tạo ngẫu nhiên
8 r1 = tf.random.normal(shape=(3, 1), mean=0., stddev=1.)
9 r2 = tf.random.uniform(shape=(3, 1), minval=0., maxval=1.)
10
```

Sử dụng tf.Variable

TensorFlow tensors là *không thể gán* (immutable). Để huấn luyện (cập nhật trọng số), ta phải dùng **Variable**.

```
1 v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
2
3 # Cap nhat gia tri
4 v.assign(tf.ones((3, 1)))
5 v[0, 0].assign(3.)
6
7 # Cap nhat cong/tru (nhu +=, -=)
8 v.assign_add(tf.ones((3, 1)))
9 v.assign_sub(tf.ones((3, 1)))
10
```

Toán học Tensor (Operations) trong TensorFlow

```
1 a = tf.ones((2, 2))
2 b = tf.square(a)    # Bình phương
3 c = tf.sqrt(a)      # Căn bậc 2
4 d = b + c           # Cộng từng phần tử (element-wise)
5
6 # Nhân ma trận (Dot product / Matmul)
7 e = tf.matmul(a, b)
8
9 # G ghép nối (Concatenate)
10 f = tf.concat((a, b), axis=0)
11
```

Tự động tính đạo hàm với GradientTape

```
1 input_var = tf.Variable(initial_value=3.0)
2 with tf.GradientTape() as tape:
3     result = tf.square(input_var) #  $y = x^2$ 
4
5 # Đạo hàm của  $x^2$  tại  $x=3$  là  $2*3 = 6$ 
6 gradient = tape.gradient(result, input_var)
7 print.gradient) # Tra về 6.0
8
```

Lưu ý: GradientTape mặc định chỉ theo dõi (watch) các `tf.Variable` có khả năng huấn luyện.

Đạo hàm cấp 2 với TensorFlow

Chúng ta có thể lồng các GradientTape để tính đạo hàm cấp cao (vd: vận tốc $\rightarrow$ gia tốc).

```
1 time = tf.Variable(0.0)
2 with tf.GradientTape() as outer_tape:
3     with tf.GradientTape() as inner_tape:
4         position = 4.9 * time**2
5         speed = inner_tape.gradient(position, time)
6
7 acceleration = outer_tape.gradient(speed, time) # 9.8
8
```

Biên dịch để tăng tốc (Just-In-Time Compilation)

Thực thi mặc định của Python (Eager Execution) khá chậm. Có thể dùng `@tf.function` để biên dịch hàm Python thành một Computation Graph (Đồ thị tính toán) tối ưu bằng XLA:

```
1 @tf.function(jit_compile=True)
2 def dense(inputs, W, b):
3     return tf.nn.relu(tf.matmul(inputs, W) + b)
4
```

Ưu điểm: Cực nhanh. *Nhược điểm:* Khó debug hơn vì mã chạy không còn là Python nguyên bản.

- **Thông trị mǎng Nghiên cứu (Research):** Hơn 70% các bài báo khoa học AI mới đều sử dụng PyTorch do sự thân thiện với lập trình viên (Pythonic).
- **Tính Pythonic cao:** Cú pháp rất gần gũi với NumPy.
- **Đồ thị tính toán Động (Dynamic Computation Graph):** Đồ thị được xây dựng on-the-fly, cực dễ chèn các lệnh `print` để debug.

Tensors trong PyTorch

```
1 import torch
2
3 # Khoi tao
4 x = torch.ones(size=(2, 1))
5 y = torch.zeros(size=(2, 1))
6 z = torch.tensor([1, 2, 3], dtype=torch.float32)
7
8 # Random (cu phap hoi khac biet)
9 r1 = torch.normal(mean=torch.zeros((3,1)), std=torch.ones((3,1)))
10 r2 = torch.rand(3, 1) # Uniform
11
```


Gán Tensor và nn.Parameter

Khác với TF, PyTorch Tensors có thể trực tiếp gán giá trị:

```
1 x = torch.zeros(size=(2, 1))
2 x[0, 0] = 1.
```

Nhưng để theo dõi đạo hàm, ta sử dụng thuộc tính `requires_grad=True`:

```
1 v = torch.ones(size=(3, 1), requires_grad=True)
2 # Hoac su dung Parameter khi viet Module
3 import torch.nn as nn
4 w = nn.Parameter(torch.rand(3, 3))
```

Tính toán Gradient bằng .backward()

Cú pháp đạo hàm của PyTorch đơn giản hơn (không cần context manager GradientTape):

```
1 x = torch.tensor(3.0, requires_grad=True)
2 result = torch.square(x)
3
4 # Goi backward de tinh dao ham cho toan bo graph
5 result.backward()
6
7 # Xem dao ham cua result theo x
8 print(x.grad) # 6.0
9
```

Tổ chức code với nn.Module

```
1 class SimpleDense(nn.Module):
2     def __init__(self, in_channels, out_channels):
3         super().__init__()
4         self.W = nn.Parameter(torch.rand(in_channels, out_channels))
5         self.b = nn.Parameter(torch.zeros(out_channels))
6
7     def forward(self, x):
8         return torch.relu(torch.matmul(x, self.W) + self.b)
9
10 model = SimpleDense(64, 32)
11
```

- $JAX = Numpy + Autograd + XLA$ (Trình biên dịch tăng tốc cực độ).
- Phổ biến ở Google DeepMind, Midjourney, Anthropic do khả năng chạy cực lớn trên cụm TPU.
- **Lập trình hàm không trạng thái (Stateless):** Mọi hàm đều trong suốt (pure functions), không sử dụng biến toàn cục. Giúp JAX an toàn khi chạy song song.

jax.numpy và PRNG Keys

JAX tái tạo 99% API của Numpy thông qua `jax.numpy`:

```
1 from jax import numpy as jnp
2 x = jnp.ones(shape=(2, 1))
3
```

PRNG Keys (Số ngẫu nhiên không trạng thái):

```
1 import jax
2 # Phai tao "chia khoa" (key) de random
3 seed_key = jax.random.key(1337)
4 r1 = jax.random.normal(seed_key, shape=(3,))
5
6 # De lay random tiep theo, phai tao key moi
7 new_key = jax.random.split(seed_key, num=1)[0]
8
```

Gán Tensor trong JAX

Giống TensorFlow, JAX tensor không thể gán trực tiếp. Phải dùng cú pháp `.at[] .set()` để trả về một tensor mới:

```
1 x = jnp.array([1, 2, 3], dtype="float32")
2 # x[0] = 10 sẽ báo lỗi
3 new_x = x.at[0].set(10)
4
```

Tính toán Gradient bằng jax.grad()

JAX sử dụng phương pháp "siêu lập trình" (hàm trả về hàm) để tính đạo hàm:

```
1 def compute_loss(input_var):  
2     return jnp.square(input_var)  
3  
4 # Truyền hàm loss qua jax.grad để lấy hàm tính đạo hàm  
5 grad_fn = jax.grad(compute_loss)  
6  
7 x = jnp.array(3.0)  
8 print(grad_fn(x)) # 6.0  
9
```

Trả về cả Loss và Gradient

Thực tế, ta thường cần cả giá trị Loss và cấu trúc Gradient. JAX cung cấp `value_and_grad()`:

```
1 def compute_loss(state, x, y):  
2     # state có thể là tuple (W, b)  
3     pass  
4  
5 # Trả về một hàm mới  
6 grad_fn = jax.value_and_grad(compute_loss)  
7  
8 # Tính toán đồng thời  
9 loss, grads = grad_fn(state, x, y)  
10
```


Biên dịch hàm cực nhanh với @jax.jit

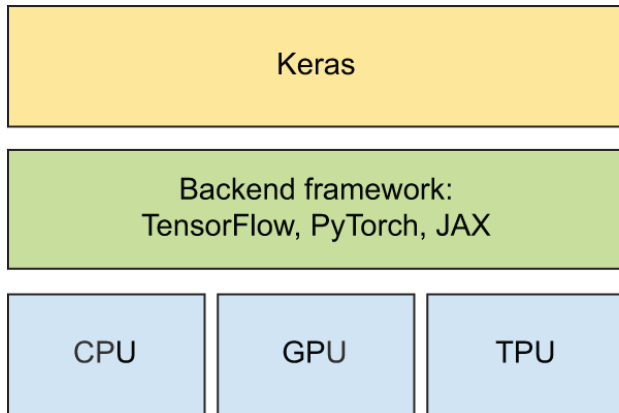
```
1 @jax.jit
2 def dense(inputs, W, b):
3     return jax.nn.relu(jnp.matmul(inputs, W) + b)
4
```

- JAX biên dịch hàm này thành mã máy XLA.
- *Ràng buộc*: Hàm phải phi trạng thái (không sửa đổi biến ngoài, cập nhật tensor phải trả về).

Keras: High-Level API cho Deep Learning

- Keras ra đời (2015) để giúp lập trình Deep Learning giống như chơi "lắp ghép LEGO".
- Keras che đi toàn bộ sự phức tạp của Low-level API (viết Gradient, vòng lặp lặp).
- Được sử dụng bởi Google, Netflix, Uber, Waymo.

Kiến trúc đa Backend của Keras



Deep learning development:
layers, models, optimizers, losses,
metrics, training loops ...

Tensor manipulation infrastructure:
tensors, variables, automatic
differentiation, distribution ...

Hardware: execution

Hình 3.1: Cấu trúc đa Backend của Keras 3.0

Layer cơ sở: Thiết kế theo hướng Đối tượng

Trong Keras, Layer là khối xây dựng cơ bản, gồm trạng thái (weights) và tính toán (forward pass).

```
1 import keras
2 class SimpleDense(keras.Layer):
3     def __init__(self, units):
4         super().__init__()
5         self.units = units
6
7     def build(self, input_shape): # Goi 1 lan duy nhat
8         self.W = self.add_weight(shape=(input_shape[-1], self.units))
9         self.b = self.add_weight(shape=(self.units,))
10
11    def call(self, inputs): # Thuc thi tinh toan
12        return keras.ops.matmul(inputs, self.W) + self.b
13
```

Lợi ích của hàm build()

Hàm build() cho phép Keras **tự động suy diễn hình dạng (shape inference)**. Bạn không cần chỉ định đầu vào dài bao nhiêu:

```
1 # Bạn không cần quan tâm lớp trước truyền vào bao nhiêu unit
2 model = keras.Sequential([
3     SimpleDense(32), # Tu dong hieu input_shape o day la 784
4     SimpleDense(64),
5     SimpleDense(10)
6 ])
7 # Goi model tren du lieu thuc te se kích hoạt build() hàng loạt
8
```

Biên dịch Mô hình (model.compile)

Để cấu hình quá trình học tập, Keras có bước compile cực kỳ chuẩn hóa:

```
1 model.compile(  
2     optimizer="adam",      # Phep toi uu gradient descent  
3     loss="mse",           # Ham mat mat  
4     metrics=["accuracy"]  # Do do hieu suat hien thi  
5 )  
6 # Train tren du lieu  
7 model.fit(x_train, y_train, epochs=10)  
8
```

Ôn tập toán học: Đường thẳng và Siêu phẳng

Mục tiêu: Xây dựng bộ phân loại tuyến tính (Linear Classifier) từ con số 0 để hiểu cơ chế lỗi của học sâu.

- Phương trình dự đoán tổng quát (Hàm tuyến tính affine):

$$y_pred = W \cdot x + b \quad (1)$$

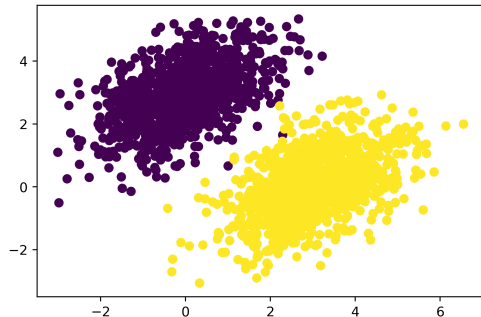
- W (Trọng số - Weights): Quyết định độ nghiêng/hướng của đường thẳng.
- b (Độ lệch - Bias): Quyết định vị trí tịnh tiến của đường thẳng so với gốc toạ độ.

Khởi tạo dữ liệu giả lập (Point Clouds)

Tạo 2 cụm điểm phân bố ngẫu nhiên trong không gian 2D:

- **Cụm 1:** Nhãn (label) = 0.
- **Cụm 2:** Nhãn (label) = 1.

Bài toán: Vẽ một đường thẳng tách biệt chính xác 2 cụm điểm này bằng Gradient Descent.

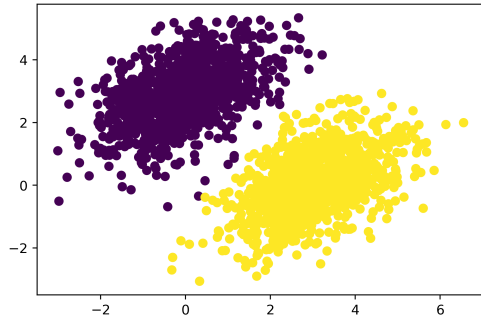


Hình 3.2: Hai cụm điểm dữ liệu ngẫu nhiên

Dự đoán ban đầu (Khởi tạo ngẫu nhiên)

Khi mô hình phân loại vừa được tạo ra:

- W, b ngẫu nhiên rất nhỏ (gần 0).
- Đường thẳng vô nghĩa, điểm MSE Loss rất cao.



Hình 3.3: Đường thẳng phân chia trước khi huấn luyện

Hàm mất mát (MSE) và Vòng lặp huấn luyện

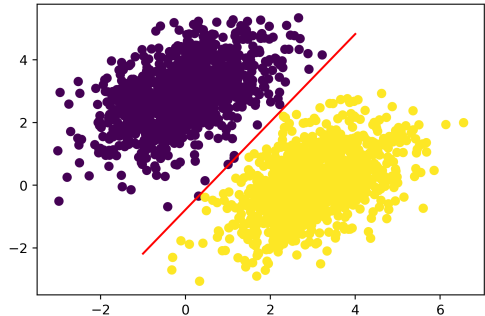
Chúng ta cập nhật W và b dựa trên đạo hàm:

```
1 learning_rate = 0.1
2 for step in range(40):
3     with tf.GradientTape() as tape:
4         predictions = tf.matmul(inputs, W) + b
5         loss = tf.reduce_mean(tf.square(targets - predictions))
6
7     grad_W, grad_b = tape.gradient(loss, [W, b])
8
9     W.assign_sub(grad_W * learning_rate)
10    b.assign_sub(grad_b * learning_rate)
11
```

Kết quả sau Vòng lặp huấn luyện

Sau nhiều bước cập nhật trọng số, bộ phân loại tuyến tính đã tự động xoay và tịnh tiến đường thẳng cắt qua khoảng trống tối ưu.

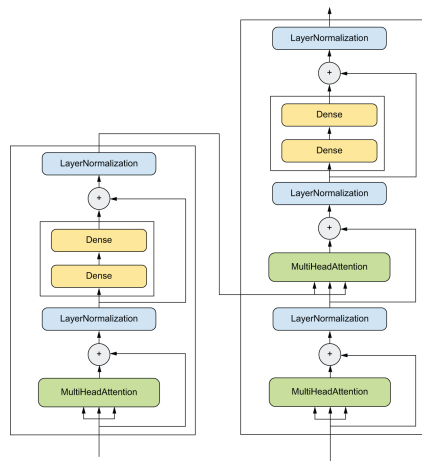
- Thuật toán Gradient Descent hội tụ!



Hình 3.4: Đường thẳng phân chia sau khi huấn luyện

Ứng dụng mở rộng: Kiến trúc phức tạp

- Mạng nơ-ron thực tế kết hợp hàng nghìn phép toán $Wx + b$ đan xen với hàm phi tuyến (ReLU, Softmax).
- Các kiến trúc khổng lồ như **Transformer** (mô hình ngôn ngữ) thực chất vẫn sử dụng nguyên lý Backpropagation và Gradient Descent này để học!



Hình 3.5: Đồ thị kiến trúc của Transformer Block

- **TensorFlow, PyTorch, JAX:** 3 "Cột trụ" Framework cung cấp sức mạnh tính toán tự động đạo hàm cực lớn trên GPU/TPU.
- **Keras:** API thượng tầng (High-level) kết nối và chuẩn hóa toàn bộ sự phức tạp của 3 Framework kia thành các khối block (Layers/Models) gọn gàng.
- **Core Logic:** Cho dù 1 lớp Dense hay 1 mô hình Transformer tỷ tham số, bản chất vẫn là Tính Loss $\rightarrow$ Lấy Gradient $\rightarrow$ Cập nhật W, b .